

City Pager / Lisbon Events Calendar

Pacote de replicacao - Inteligência — Infraestrutura Deep Dive

Terminus — Intelligence Infrastructure

Documento: 6 de 6 — Infraestrutura de Inteligência (deep dive)

Audiência: quem precisa de compreender o stack AI ponta a ponta

Fontes de código: pipeline/intelligence/*, pipeline/qualification/*, catalog sinks, Tier 5/6, publish gates, For You

Este documento descreve o stack completo de inteligência tal como está implementado no repositório. Formas curtas canônicas: 02-pipeline-e-inteligencia.md e docs/PIPELINE.md. Aqui expandimos num único quadro end-to-end.

1. O que significa “intelligence” no Terminus

Terminus tem dois problemas de inteligência distintos — não confundir:

Camada | Pergunta | Onde vive | AI?

Ingestion intelligence | “Que posts IG são eventos futuros reais, quais os campos, que venue/promoter, e é seguro publicar?” | pipeline/ | LLM / vision / OCR / Whisper / web verify

Discovery intelligence | “Que eventos já publicados deve *este utilizador* ver a seguir?” | src/lib/recommendationEngine.ts | Não — scoring por regras (rules_v1)

Quase tudo a que se chama “AI pipeline” é a camada de ingestão. Venues e promoters entram nesse stack de três formas:

1. Como fontes da watchlist — handles activos dizem ao Apify *o que scapar*.
2. Como alvos de resolução — strings de venue extraídas têm de mapear a um venue_id canónico.
3. Como candidatos de crescimento — nomes de venue desconhecidos e @mentions viram propostas humanas de catálogo.

O sistema é deliberadamente evidence-bound e gate-heavy. Modelos propõem; confiança calculada + regras duras + aprovação humana opcional decidem o que vai ao ar.

2. Filosofia (porque o pipeline é assim)

Cinco compromissos de desenho moldam cada tier:

1. Evidence over invention

Campos críticos (start_datetime, venue, price, is_free, ticket URL, age) têm de aparecer na caption, slide, OCR ou metadata. Os prompts proíbem inventar covers de clube, 18+, ou domínios placeholder de bilhetes.

2. Model self-score is not the gate

Os LLMs emitem confidence_score, mas o auto-publish usa calculated-confidence.ts: evidência ponderada por campo, menos penalties de conflito/inferência. O score do modelo é só um prior limitado (≤ 0.85 , blend 25%).

3. Never promote a promoter to a venue

Se um promoter posta um evento no Lux, o fallback do owner handle não pode inventar o nome do promoter como venue. Codificado em venue-resolve.ts.

4. Vision is expensive — trigger, don't always run

Caption (Tier 1) corre sempre em posts mantidos. Vision de carousel/vídeo só quando faltam campos

obrigatórios (ou --force-vision).

5. Verify suggests; humans (or clean verify) authorize publish

Tier 5 nunca edita linhas Processed. Publish exige segurança mecânica e (publish_auth=human_approved ou Tier 5 verified limpo sem suggested_corrections).

3. Mapa end-to-end

```

CATALOG (Venues + Promoters)
  Sheets and/or Supabase (CATALOG_SOURCE)
  instagram_handle + is_active = scrape watchlist
    |
    v readPipelineWatchlist()
SCRAPE (Apify Instagram post scraper)
  -> EventsRawRow -> image archive -> pipeline_posts (status=new)
    |
    v processPost() per row
TIER 0 Pre-filter -> discard recaps/memes/non-events
TIER 1 Caption LLM -> structured events[]
TRIGGER mandatory fields incomplete?
TIER 3 Carousel VL (+ optional DocAI OCR)
  or TIER 4 Video frames + Whisper
MERGE caption <-> vision (min conf - penalties)
RECONCILE same-night lineup / slide dupes
REPAIR 24:00, overnight end, bad URLs, floor qualifiers
RESOLVE venue -> calculated confidence -> VALIDATE
CATALOG CANDIDATES for unresolved venues + unknown @mentions
  |
  pass |          review / soft fail
    v          v
Processed Events (staging) pipeline_review_queue
publish_auth = " -> /admin/event-review (Tier 6)
  |
  v
TIER 5 online verify (suggestions only)
  |
clean verified --+  unclean / disputed --> review queue
    v
npm run publish -> isPublishAuthorized -> Events Clean New CSV
    v
Next.js loaders -> Calendar / For You (rules_v1 scorer)
  
```

Jobs longos nunca correm no Vercel. Admin enfileira pipeline_runs; um npm run worker local faz claim e executa scrape/extract/verify/full/profile-images.

4. Seleccionar *o que* observar: venues & promoters como fontes

4.1 Watchlist SoT

O scrape não é “todo o Instagram”. É uma lista curada de handles Instagram activos tipados como venue ou promoter.

Ordem de resolução (readPipelineWatchlist em pipeline/sinks/catalog-store.ts):

CATALOG_SOURCE / WATCHLIST_SOURCE | Comportamento

Supabase preferido (auto/supabase) | venues + promoters activos com handles

Senão Sheets Venues + Promoters | Mesmos campos

Fallback de cutover (WATCHLIST_SOURCE=auto + catálogo vazio) | Tabs legacy Fontes IG

Cada entrada da watchlist carrega:

- handle normalizado
- name de display
- type: venue | promoter — crítico no venue resolve
- is_active — inactivos não são scrapados

UI admin: /admin/venues, /admin/promoters, /admin/scrapers. Check antes de flip de sources: npm run diff-watchlist.

4.2 Porque o type importa

Durante extracção, process-post.ts faz lookup de sourceType do owner do post. Esse bit controla:

- Se owner-handle fallback é permitido quando não há string de venue (venues sim, promoters nunca).
- Se ruído “venue = promoter handle” é ignorado ao propor catalog candidates.
- Naming de display venue quando o nome resolvido está vazio.

Esta é a primeira fronteira de inteligência entre “sítios” e “organizadores”.

4.3 Modo profile-images (não é AI de eventos)

Actor Apify profile separado sincroniza avatares para Supabase venue-images (+ opcional Sheets primary_image_url). Não extrai eventos. full = scrape posts + extract (+ Tier 5); não inclui profile-images.

5. Scrape → posts raw (input da AI)

Actor: apify/instagram-post-scraper (id default nH2AHrwxETRJoN5hX).

Fluxo:

1. Carregar watchlist activa (opcionalmente filtrar --handle=).
2. Cutoff incremental do último scrape, ou --max-age-days.
3. Run Apify (batch ou per-account via PIPELINE_RUN_MODE).
4. Transform → EventsRawRow (scrapers/instagram-transform.ts): caption, media type, carousel URLs, location, mentions, hashtags, links, owner, posted_at, etc.
5. Arquivar imagens display via API persist-image da app.
6. Upsert pipeline_posts com processing_status = new.

Cada post é uma unidade de auditoria: o status passa por new → discarded | needs_review | processed, com cada tier AI escrito em pipeline_extractions quando postDbId está definido.

6. Orquestrador por post (process-post.ts)

Este ficheiro é a espinha dorsal. Tratar como máquina de estados:

```
pre_filter -> [discard?]  
broad_llm -> [vision?]  
merge -> reconcile  
for each event:  
  auto_repair -> venue_resolve -> calculate_confidence -> validate
```

-> catalog candidates if unresolved
-> past_event discard | needs_review | Processed row

Artefactos persistidos (quando há id Supabase do post):

Chave de tier | Conteúdo

pre_filter | keep/discard + post_pattern
caption | eventos Tier 1
ocr | texto DocAI por slide (opcional)
video_transcript | texto Whisper (opcional)
vision | eventos Tier 3/4
merge | eventos merged + conflicts
validation | status por evento, reasons, venue method, calculated conf

7. Tier 0 — Pre-filter (seleccionar *posts*, ainda não eventos)

Módulo: intelligence/pre-filter.ts

Modelo: PIPELINE_TEXT_MODEL (default gpt-4o-mini)

Objectivo: Parar de pagar extracção em memes, merch e recaps.

Gate determinístico (sem LLM)

- Caption < 20 caracteres e media não é carousel → discard como not_event / caption_too_short.
- Carousels estão isentos: programas mensais muitas vezes põem tudo nos slides.

Classificação LLM

JSON validado com Zod:

- is_event_post: boolean
- confidence: 0–1
- post_pattern:
 - single_event | multi_event | monthly_program | announcement | recap | not_event
- reason curto

Regra de discard: is_event_post ou pattern recap ou not_event.

Fail-open em erro de parse: classificador ilegível → keep como single_event com confidence 0.5. Preferir falsos positivos (custo de review) a dropar eventos reais em silêncio.

announcement e monthly_program são mantidos: ainda podem render eventos (ou precisar de vision). O pattern também alimenta soft gates posteriores (ex. program_undersplit se monthly_program mas só um evento extraído).

8. Tier 1 — Broad caption extraction

Módulo: intelligence/broad-event-extraction.ts

Corre sempre em posts mantidos.

Inputs ao modelo

JSON: caption (até 6k chars), owner username/full name, IG location_name, posted_at, hashtags, external links.

Contrato de output (Zod)

Um ou mais eventos com campos alinhados ao SCHEMA: title, descriptions, category, tags (≤ 5), start/end ISO com offset Europe/Lisbon, venue_name_raw, prices, is_free (tri-state: true/false/omit), ticket URL, age, confidence_score.

Lei do prompt (evidence-bound)

- Datas relativas em português resolvidas contra posted_at (próxima ocorrência futura).
- Vários eventos distintos → várias entradas no array.
- Recorrente (“todas as quintas”) → próxima ocorrência única + nota em extraction_notes.
- Nunca inventar tickets, age, prices, ou free-by-default.
- Preço desconhecido → deixar is_free unset (badge Free na UI só quando true).
- events: [] vazio se nada attendable/upcoming.

Falha de parse → lista de eventos vazia (downstream pode disparar vision via no_caption_events).

Cada evento fica com extraction_source: 'caption'.

9. Vision trigger (campos obrigatórios)

Módulo: qualification/mandatory-fields.ts

Vision não corre só porque o media é carousel. Corre quando:

Condição | Reason string

--force-vision | force_vision

Caption devolveu zero eventos | no_caption_events

Algum evento caption sem title, start_datetime válido, ou venue | event[i] missing ...

Venue pode ser satisfeito por IG location_name mesmo se a caption omitiu venue_name_raw.

Se media é video e flags de vídeo activas → Tier 4; senão → caminho Tier 3 carousel/imagem. Se vídeo mas flags off → vision skipped (logado em tiersRun).

10. Tier 3 — Carousel / image vision

Módulo: intelligence/carousel-event-vision.ts

Porquê existe: Programas mensais de Lisboa põem datas nos slides 2...N, não na caption. Este é o pass caro de maior ROI.

Pipeline dentro do Tier 3

1. Resolver URLs de slides (máx 10): Apify carousel_slide_urls → fallback HTML embed Instagram → single display/thumbnail.
2. Arquivar URLs CDN efémeras para storage durável (salvo skipArchive).
3. OCR opcional (Google Document AI por slide) → persistido como tier ocr. OCR é confiado para caracteres; vision para layout/associação.
4. LLM multimodal em chunks de 2 slides (Nemotron VL via NVIDIA NIM por default, fallback OpenAI gpt-4o).
5. Merge dos chunks em events[] com:
 - source_slide_indices (1-based)
 - on_slide_text_evidence — texto de data/hora verbatim (obrigatório quando se afirma datetime)

Mesmas regras evidence-bound da caption, mais abreviaturas de flyers PT (SEX, 22H, MAR, ...). Carousels de programa devem emitir um evento por dia/evento distinto, não um mega-evento colapsado.

11. Tier 4 — Video

Módulo: intelligence/video-event-extraction.ts

Activado por flags env (PIPELINE_VIDEO_FRAMES / PIPELINE_VIDEO_WHISPER).

Caminho típico: ffmpeg amostra ~4 frames → transcript Whisper (tier video_transcript) → vision multimodal sobre frames + contexto do transcript. Mesmo schema estruturado do Tier 3.

12. Merge — arbitragem caption ↔ vision

Módulo: intelligence/merge-extractions.ts

Regra central

Confidence final = $\min(\text{caption}, \text{vision}) - 0.15 \times \text{conflict_count}$, nunca max. Desacordo tem de baixar as odds de auto-publish.

Campos de conflito

date (dia de calendário), venue (nome normalizado), price (delta $\geq 1\text{€}$), is_free.

Heurísticas de preferência de campo

Campo | Preferência

Datetime | Vision se caption sem data ou $\text{conf caption} < \text{PIPELINE_MERGE_CAPTION_DATETIME_THRESHOLD}$ (default 0.8)

Title | Caption, salvo title genérico (Event, Agenda, ...) e vision melhor

Venue / price / free / ticket | Caption primeiro, preencher gaps com vision

Tags | União, capped

Source | extraction_source: 'merged'; manter slide evidence da vision

Program split

Se vision encontra mais eventos que caption e caption tem exactamente um → o split da vision ganha (caption usada como seed de baixa confiança por evento vision). Caso clássico de monthly-program.

Tier-empty disagreement

Um lado tem eventos, o outro nenhum → flag tierEmptyDisagreement (depois: review + penalty de confiança).

13. Reconcile — colapsar lineups da mesma noite

Módulo: qualification/reconcile-post-events.ts

Vision muitas vezes emite um “evento” por slide de artista para a mesma noite. Reconcile faz match fuzzy da mesma occurrence (title/time/venue) e faz merge:

- Preferir title descritivo mais longo; lineup na description/tags.
- Start mais cedo, end mais tarde.
- Confidence = min das partes.

- Mega-spans (>36h) clamped/flagged para validação.

Corre antes do fingerprint para não publicar cinco “eventos” para uma noite de clube.

14. Auto-repair — salvage determinístico

Módulo: qualification/auto-repair.ts

Corre antes da validação. Preferir fix mecânico a rejeitar:

Repair | O quê

fixed_24h_time | 24:xx ilegal → dia seguinte 00:xx

overnight_end_rollover | End antes do start → end +1 dia (noites de clube)

cleared_placeholder_ticket_url | example.com / picsum / ...

cleared_free_price_conflict | is_free + preço positivo

dropped_zero_duration_end | Start/end idênticos → dropar end

stripped_venue_floor_qualifier | “Lux - 1º andar” → “Lux”; “Sótão” puro limpo

Estas repairs ficam no artefacto de validação para auditoria.

15. Venue resolution — seleccionar o *sítio*

Módulo: qualification/venue-resolve.ts

Índice: src/data/venueIndex.ts (aliases, handles, names).

Ordem de load do índice

1. Supabase venues se CATALOG_SOURCE permite e não-vazio
2. Senão Sheets/CSV Venues
3. Senão sintetizar de Fontes legacy (só cutover)

Ordem de resolve para um evento extraído

1. extracted venue_name_raw -> resolve by name/alias
if string present but unresolved -> STOP unresolved
(NEVER fall back to owner)
2. else IG location_name -> resolve
3. else if sourceType === promoter -> unresolved (sourceAsVenueRisk)
4. else if venue account and no extracted/location string
-> resolve by owner Instagram handle
5. location present but failed + owner available
-> still unresolved (do not override a different named place with owner)

Em sucesso, enrichment copia city, neighborhood, venue_address do catálogo. City no Processed prefere a city resolvida; não hardcode Lisboa quando conhecida.

Soft gate source_as_venue

Quando resolve falha e a lógica detecta risco promoter/owner-as-venue, a validação emite source_as_venue → review, não auto-pass.

Regra dura de produto

Venue unresolved bloqueia auto-pass (venue_unresolved). Tem de haver human review ou crescimento de catálogo primeiro.

16. Crescer o catálogo — seleccionar *novos* venues & promoters

Módulo: pipeline/sinks/catalog-candidates.ts

UI: /admin/catalog-candidates

Separado da fila de review de eventos.

Durante processPost, duas streams de sightings:

A. @mentions desconhecidas → promoter candidates

- Parse caption/mentions.
- Skip owner e qualquer handle já em venues + promoters.
- Upsert candidate pending kind: 'promoter' com evidência (URL do post, snippet da caption).

B. Strings de venue unresolved → venue candidates

- Após resolve falhado (e não past_event).
- Skip nomes lixo (unknown, lisbon, URLs, ...) via isPlausibleCatalogName.
- Skip ruído promoter-owner-as-venue.
- Upsert kind: 'venue' com sample_venue_name_raw, suggested city, etc.

Candidates usam identity_key (h:handle ou n:normalized_name). Sightings repetidos bump rows pending; approved/rejected ficam fechados até reopen. Humanos aprovam no catálogo Supabase (reviewer pode flip venue↔promoter).

É assim que a inteligência selecciona candidatos para o catálogo, enquanto humanos permanecem a autoridade do que entra no scrape graph e no venue index.

17. Calculated confidence

Módulo: qualification/calculated-confidence.ts

Substitui confiar no self-report do modelo como gate de publish.

Scores por campo (depois ponderados)

Campo | Alto | Baixo | Peso

title | len ≥ 4 → 1 | else 0.2 | 0.20

start | ISO válido → 1 | else 0 | 0.25

venue | raw presente → 0.8 | else 0.2 | 0.20

price | free/paid conhecido → 0.7 | unknown 0.4 | 0.10

ticket | URL boa 0.6 / má 0.1 / ausente 0.5 | 0.05

evidence | slide text / caption / other | 1 / 0.7 / 0.5 | 0.20

Blend: $0.75 * evidenceScore + 0.25 * \min(modelConf, 0.85)$.

Penalties

- -0.12 por merge conflict

- -0.20 tier-empty disagreement
- -0.25 critical fields inferred (heurística: notas do modelo dizem infer/assume/guess e age/ticket/price/free presentes)

Resultado sobrescreve event.confidence_score antes da validação. Threshold: PIPELINE_PUBLISH_CONFIDENCE_THRESHOLD (default 0.7).

18. Validation — pass / review / fail

Módulo: qualification/validate-event.ts

Hard fail → caminho discard (não fila humana)

- missing_title
- missing_or_invalid_start_datetime
- past_event (também short-circuit: nunca abre review)

Se todos os candidatos dum post são só past → status do post discarded.

Soft → needs_review (Tier 6)

Inclui: venue_unresolved, source_as_venue, low_confidence, tier_conflict, tier_empty_disagreement, critical_field_inferred, bad_ticket_url, end_before_start, implausible_duration, program_undersplit, outside_service_area (city fora da metro de Lisboa), conflito price/free, etc.

Pass → linha Processed Events

Requer: title, start futuro válido, venue raw, venue resolved, confidence $\geq$ threshold, sem soft reasons. Constrói fingerprint, normaliza category, põe publish_auth: " (não autorizado até verify/human).

Fingerprint (estilo djb2):

source_post_id | YYYY-MM-DD | HH:mm(bucket 30min UTC) | title_norm | venueKey

Dedupe em batch fica com a maior confidence; drop se já existe em Processed.

19. Category normalization

Módulo: qualification/normalize-category.ts

Mapeia categories free-form do modelo para o conjunto fechado do produto (music, nightlife, art, ...) antes de escrever na sheet. Mantém filtros do calendário coerentes.

20. Tier 5 — Online verification

Módulo: intelligence/event-verification.ts

Modelo: PIPELINE_VERIFY_MODEL (default gpt-4o) + Brave Search opcional, senão OpenAI Responses + web_search_preview.

Corre após extract em full/extract salvo --skip-verify. Também npm run verify sozinho.

Papel

Sugestões de fact-check para um humano (ou para lógica authorize-or-queue). Nunca escreve correcções em Processed.

Verdicts

Verdict | Significado

verified | Fontes independentes confirmam title/date/venue (± 1 dia / aliases OK)

disputed | Contradição, cancelamento, evento errado — pode incluir suggested_corrections

not_found | Sem menção independente (só IG não conta)

inconclusive | Evidência fraca/mista

Clean vs unclean

isCleanVerification = verdict === verified e suggested_corrections vazio.

- Clean → pode ficar em Processed e depois autorizar publish.
- Unclean / disputed / corrections → pipeline_review_queue com contexto de verificação.

Persistido em pipeline_verifications para auditoria.

21. Tier 6 — Human review

UI: /admin/event-review sobre pipeline_review_queue.

Operadores veem caption, media, tier trail, validation reasons, sugestões Tier 5. Acções:

- Approve → append Processed com publish_auth=human_approved (pode aplicar field fixes sugeridos).
- Reject → fecha o item.

Feedback de qualidade pode ir para event_review_feedback (migration 018). Scripts de recovery: expire-review-queue, re-resolve-review-queue, unresolved-venues-report, quarantine-publish-unsafe.

Esta é a autoridade final de inteligência quando a automação está insegura — sobretudo identidade de venue e datas disputadas.

22. Publish gate — seleccionar o que vai *live*

Módulo: qualification/publish-safe.ts

Comando: npm run publish (Processed → Events Clean New, só rows novas).

Duas camadas:

Mecânico isPublishSafe

Title, start futuro, venue_id resolved, end sano, sem bad ticket URL, sem free+price conflict. Defense-in-depth para rows staging inseguras não shipparem.

Autorização isPublishAuthorized

Mecanicamente safe E:

- publish_auth === human_approved, ou
- event id está no set clean-verified do Tier 5.

Rows de auto-pass deixam publish_auth vazio até uma dessas condições. É assim que Terminus separa “parece extractável” de “permitido no calendário público”.

Feed live: CSV publicado → NEXT_PUBLIC_EVENTS_CSV_URL → data/loaders → eventsAdapter → Calendar / For You.

23. Storage & observabilidade da inteligência

Store | Papel na inteligência

pipeline_posts | Unidade IG raw + processing status
pipeline_extractions | JSON/raw de cada tier para replay & admin drawer
pipeline_review_queue | Soft failures + unclean verifies
pipeline_verifications | Auditoria Tier 5
pipeline_runs / pipeline_config | Job queue + worker heartbeat
pipeline_catalog_candidates | Venues/promoters propostos
Sheets Processed Events | Staging calendar rows
Sheets Events Clean New | SoT público da app
Supabase venues / promoters | Catalog SoT após mig 025 + seed

Superfícies admin: /admin/venues, /admin/promoters, /admin/events-raw (tier trail), /admin/event-review, /admin/processed, /admin/catalog-candidates, /admin/scrapers, /admin/bugs.

Self-test de gates: pipeline → npm run test:gates.

24. Modelos & superfície env

Pass | Default / serviço

Tier 0 / 1 text | gpt-4o-mini (PIPELINE_TEXT_MODEL)
Vision | NVIDIA Nemotron VL → fallback gpt-4o
Whisper | whisper-1
OCR | Google Document AI (opcional)
Tier 5 | gpt-4o + Brave opcional
Scrape | Apify post + profile actors

Credenciais mínimas: Apify, OpenAI, Supabase service role, Sheets SA. Vision NVIDIA:
NVIDIA_NIM_API_KEY.

25. Inteligência de produto: For You (não LLM)

Módulo: src/lib/recommendationEngine.ts

Versão: rules_v1 (não mudar pesos sem nova versão de algoritmo).

Esta camada selecciona eventos para um utilizador a partir do CSV já publicado, usando sinais sociais/prefs:

Sinal | Peso approx

Followed venue | +10
Followed promoter | +8
Persona match | +6
Friend going | +5
Saved tag | +4
Liked category | +3
Free preference | +2
High-energy boost | +2
Cold start | random upcoming

Friends são mútuos (friend_requests). Follows de venue/promoter são prefs privadas de discovery, não um

grafo público de followers (docs/FRIENDS_VS_FOLLOWS.md). Telemetry é aditiva e não deve mudar a ordem dos scores (docs/RECOMMENDATION_TELEMETRY.md).

Portanto: pipeline AI constrói o catálogo de verdade; For You faz o rank. Inteligências diferentes, produto partilhado.

26. Como as três entidades são “seleccionadas” — resumo

Events

1. Post da watchlist scrapado.
2. Tier 0 mantém só posts provavelmente de eventos futuros.
3. Tier 1 (+ opcional 3/4) extrai candidatos estruturados.
4. Merge/reconcile/repair produzem eventos ao nível da occurrence.
5. Confidence + validation fazem route pass vs review vs discard.
6. Tier 5/6 autorizam publish.
7. For You opcionalmente re-ranka por utilizador.

Venues

1. Humano (ou seed) mantém catálogo com handles/aliases.
2. Handles activos de venue são scrapados como fontes.
3. Strings de venue extraídas resolvem contra o índice (regras estritas de owner-fallback).
4. Falhas viram venue catalog candidates + event review.
5. Venues aprovados reforçam resolve futuro e cobertura de scrape.

Promoters

1. Mesmo caminho de catálogo/watchlist que venues, tipados promoter.
 2. Os posts são scrapados, mas não podem tornar-se a venue do evento via owner fallback.
 3. @mentions desconhecidas viram promoter candidates.
 4. Utilizadores seguem promoters para For You (+8); follows não mudam a extracção do pipeline.
-

27. Failure modes que a infraestrutura foi construída para prevenir

Failure | Mitigation

Recap posts como eventos futuros | Tier 0 discard recap
Datas/preços/tickets inventados | Prompts evidence-bound + calculated conf + critical_field_inferred
Nome de promoter como venue | venue-resolve + source_as_venue
Um programa → um mega-evento | Vision split + program_undersplit
Artist slides → N noites duplicadas | reconcile-post-events
Auto-publish em verify fraco | isPublishAuthorized
Drop silencioso de bons posts | Pre-filter fail-open em parse error
CDN image rot | Arquivar slides/imagens antes da vision
Processed leftovers inseguros | isPublishSafe no publish + quarantine scripts
Ruído fora da metro de Lisboa | review outside_service_area

28. Mapa de módulos (índice de código)

Path | Papel

pipeline/process-post.ts | Orquestração por post
pipeline/cli/run.ts / cli/worker.ts | Modes + job queue
pipeline/intelligence/pre-filter.ts | Tier 0
pipeline/intelligence/broad-event-extraction.ts | Tier 1
pipeline/intelligence/carousel-event-vision.ts | Tier 3
pipeline/intelligence/video-event-extraction.ts | Tier 4
pipeline/intelligence/docai-ocr.ts | OCR
pipeline/intelligence/vision-client.ts | Clientes OpenAI / NIM
pipeline/intelligence/merge-extractions.ts | Caption↔vision
pipeline/intelligence/event-verification.ts | Tier 5
pipeline/qualification/mandatory-fields.ts | Vision trigger
pipeline/qualification/auto-repair.ts | Fixes determinísticos
pipeline/qualification/reconcile-post-events.ts | Collapse de lineup
pipeline/qualification/calculated-confidence.ts | Confiança real
pipeline/qualification/validate-event.ts | Hard/soft gates
pipeline/qualification/venue-resolve.ts | Venue canônico
pipeline/qualification/dedupe.ts | Fingerprints
pipeline/qualification/publish-safe.ts | Gate de publish live
pipeline/qualification/normalize-category.ts | Categories
pipeline/sinks/catalog-store.ts | Watchlist + catalog SoT
pipeline/sinks/catalog-candidates.ts | Propostas de novos venue/promoter
src/lib/recommendationEngine.ts | For You rules_v1
src/lib/adminEventReviewFeedback.ts | Loop de qualidade Tier 6

29. Operar o stack de inteligência (dia-a-dia)

```
cd pipeline && npm install
# configurar pipeline/.env
npm run worker          # terminal dedicado
npm run full -- --limit=10 --max-age-days=14
npm run publish         # após clean verifies / human approvals
```

Ou enfileirar em /admin/scrapers com worker heartbeat verde. Flags úteis: --handle=, --force-vision, --skip-verify, --dry-run, --requeue, --from-apify-run=.

Golden replay (só report): npm run golden. Regressão de gates: npm run test:gates.

30. Essência num parágrafo

A infraestrutura de inteligência do Terminus é um extractor CAF-style, evidence-bound, multi-tier sobre uma watchlist Instagram curada de venues e promoters: gates de texto baratos descartam lixo; caption LLM extrai; vision/OCR/Whisper disparam só quando faltam campos obrigatórios; merge prefere honestidade a optimismo; repair e reconcile determinísticos limpam a confusão de noites de clube; venue resolve recusa promoter-as-venue; confiança calculada e hard gates separam auto-Processed de human review; Tier 5 sugere correcções web sem auto-editar; publish só envia rows mecanicamente safe e verified-or-human-approved para Events Clean New; em separado, um scorer For You não-ML faz rank desse catálogo live para cada utilizador. Catalog candidates fecham o loop para sítios unresolved e handles desconhecidos crescerem a watchlist sob controlo humano.